UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
CENTRO UNIVERSITARIO DE OCCIDENTE
DIVISIÓN DE CIENCIAS DE LA INGENIERÍA
CURSO: ORGANIZACIÓN DE LENGUAJES Y COMPILADORES 2
CATEDRÁTICO: ING. MARIO RAMÍREZ



PROYECTO 2: St4ck Over-Pig
MANUAL DE TECNICO

ESTUDIANTE **CARNÉ:**
Cristian Alejandro Roldán López **202147280**

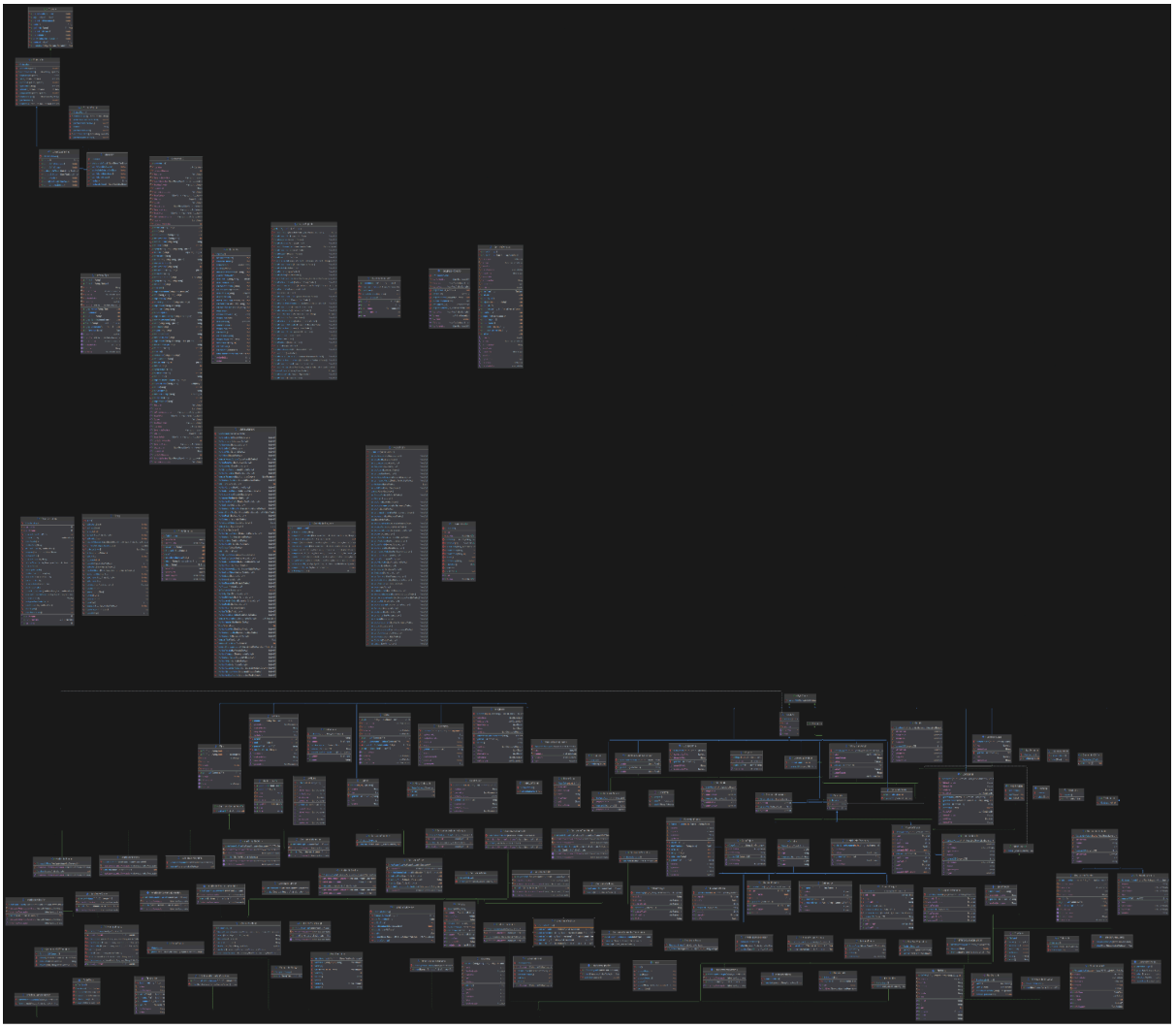
QUETZALTENANGO, 27 DE OCTUBRE 2,026

Tecnologías Utilizadas.

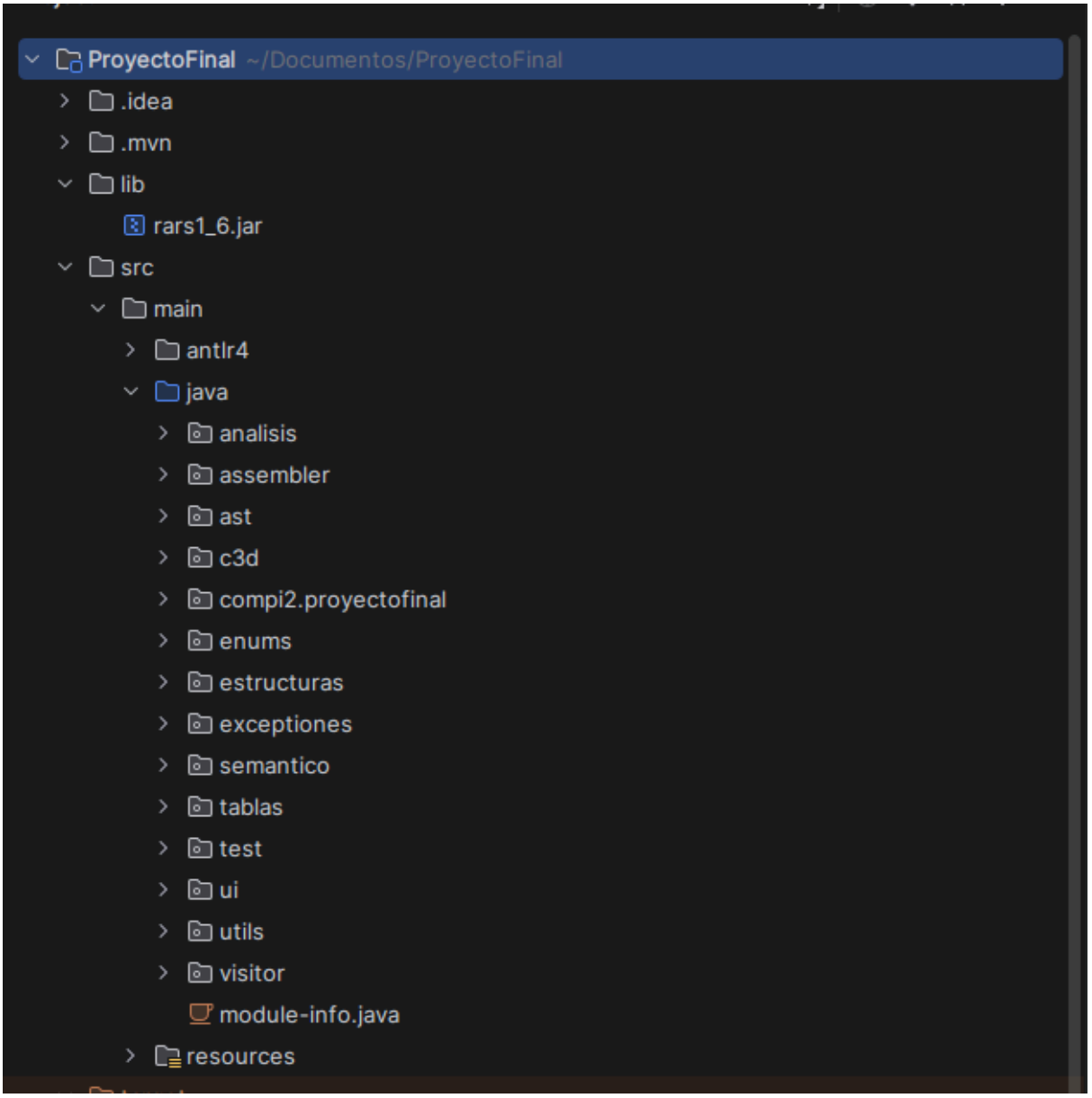
Las tecnologías utilizadas en la práctica fueron desarrolladas en :

- **JDK: 21**
- **IDE: Intellij IDEA Ultimate**
- **Framework: JavaFX 21**
- **ANTLR4**

Diagrama de clases



Distribución del proyecto



La distribución del proyecto se realizó de la siguiente manera:

- **antrl4:** Cuenta con las gramáticas utilizadas para la práctica, son tres archivos llamados: GrammarPigLatin.g4, GrammarPython.g4, GrammarZetariano.g4
- **analisis:** Se une todo lo realizado en la clase CompilarCodigo para armar el AST y Pila, en AnalisisErrores se reconocen los errores léxicos y sintacticos

que puedan existir.

- **ast:** Se realiza la construcción del árbol AST con Visitor, nodo representa una clase con los atributos que tendrá cada una.

- **enums:** Cuenta con tipos de datos que existen en el lenguaje, tipos de nodos que existen, tipo de operaciones que utilizara la pila y los tipos de símbolos.

- **estructuras:** Acá se encuentra la implementación de la Tabla Hash que nos permite guardar los símbolos utilizados en la tabla de símbolos, Pila que nos sirve para crear la pila de ámbitos y la pila de llamadas.

- **c3d:** Acá se maneja la generación de código de tres direcciones, generando el código a C e implementando la interfaz para generar el código.

- **semantico:** Cuenta la clase AnalisisSemantico donde se validaron las reglas semánticas del lenguaje.

- **tablas:** Se encuentra la tabla de símbolos utilizada.

- **ui:** Esta dividida en button_option donde se encuentran las vistas de los botones y las acciones que pueden realizar, view donde se visualiza todo y se une el contenido de button_option.

- **visitor:** Visitor de los tres lenguajes implementados.

Palabras reservadas y símbolos.

```
Lenguaje .Y
/*LEXER*/

WS: [ ]+ -> skip;

NEWLINE: ('\r'? '\n' | '\r') [ \t]*;

COMENTARIO_LINEA: '//' ~[\r\n]* -> skip;
COMENTARIO_BLOQUE: '/*' .*? '*/' -> skip;
COMENTARIO_HASH: '#' ~[\r\n]* -> skip;

INCREMENTO: '++';
DECREMENTO: '--';

MAS: '+';
RESTA: '-';
MULTIPLICACION: '*';
DIVISION: '/';

MAYORIGUAL: '>=';
MENORIGUAL: '<=';
COMPARACION: '==';
DIFERENCIA: '!=';
MENOR: '<';
MAYOR: '>';

AND: '&&';
OR: '||';
NEGACION: '!';

COMILLAS: '"' (ESC | ~["\\])* '"';
fragment ESC: '\\' . ;
COMILLASSIMPLES: '\'' ~['\r\n] '\'';
```

DECIMAL: [0-9]+ '.' [0-9]+;
NUMERO_ENTERO: [0-9]+;

ENTERO: 'entero';
FLOTANTE: 'flotante';
CARACTER: 'caracter';
BOOL: 'bool';
CADENA: 'cadena';

VERDADERO: 'verdadero';
FALSO: 'falso';

SI: 'si';
ENTONCES: 'entonces';
SINO: 'sino';
CONTRARIO: 'contrario';

ELEGIR: 'elegir';
CASO: 'caso';
ROMPER: 'romper';
SIEMPRE: 'siempre';

PARA: 'para';
CONTINUAR: 'continuar';
MIENTRAS: 'mientras';
HACER: 'hacer';

IMPRIMIR: 'imprimir';
LEER: 'leer';

SECCION_ESTRUCTURA: '%estructuras';
ESTRUCTURA: 'estructura';

SECCION_FUNCION: '%funciones';
DEFINIR: 'definir';
RETORNO: 'retornar';
TIPO_RETORNO: '->';

ID: [a-zA-Z_][a-zA-Z_0-9]*;

LLAVE_ABRE: '{';
LLAVE_CIERRA: '}';
CORCHETE_ABRE: '[';
CORCHETE_CIERRA: ']';
PARENTESIS_ABRE: '(';
PARENTESIS_CIERRA: ')';

PUNTO_COMA: ';';
PUNTO: '.';
COMA: ',';
DOS_PUNTOS: ':';
IGUAL: '=';

INDENT: '\u0002INDENT_NUNCA_COINCIDE\u0002';
DEDENT: '\u0002DEDENT_NUNCA_COINCIDE\u0002';

Gramática

/*GRAMATICA*/

program:
    NEWLINE*
    seccionEstructuras?
    NEWLINE*
    seccionFunciones
    NEWLINE*
    EOF
    ;

```
seccionEstructuras:
    SECCION_ESTRUCTURA NEWLINE+
    declaracionEstructura
    (NEWLINE* declaracionEstructura)*
    ;

declaracionEstructura:
    ESTRUCTURA ID DOS_PUNTOS
    NEWLINE+
    INDENT
    campoEstructura
    (NEWLINE+ campoEstructura)*
    NEWLINE*
    DEDENT
    ;

campoEstructura:
    tipo ID dimension?
    ;

seccionFunciones:
    SECCION_FUNCION NEWLINE+
    declaracionFuncion+
    ;

declaracionFuncion:
    DEFINIR ID PARENTESIS_ABRE listaParametros? PARENTESIS_CIERRA
(TIPO_RETORNO tipo)? DOS_PUNTOS bloque
    ;

listaParametros:
    parametro (COMA parametro)*
    ;
```

```
parametro:
      CORCHETE_ABRE CORCHETE_CIERRA tipo ID   # parametroArreglo
    | LLAVE_ABRE LLAVE_CIERRA tipo ID         # parametroEstructura
    | tipo ID                                 # parametroSimple
    ;

bloque:
    NEWLINE+
    INDENT
    (sentencia | NEWLINE)*
    DEDENT
    ;

sentencia:
      declaracionVariable NEWLINE          # sentDeclaracionVariable
    | asignacion NEWLINE                   # sentAsignacion
    | llamadaFuncion NEWLINE               # sentLlamadaFuncion
    | accesoVariable (INCREMENTO | DECREMENTO) NEWLINE   #
sentIncrDecrPostfijo
    | (INCREMENTO | DECREMENTO) accesoVariable NEWLINE   #
sentIncrDecrPrefijo
    | imprimirStmt NEWLINE                 # sentImprimir
    | leerStmt NEWLINE                     # sentLeer
    | RETORNO expresion? NEWLINE           # sentRetorno
    | ROMPER NEWLINE                       # sentRomper
    | CONTINUAR NEWLINE                    # sentContinuar
    | declaracionEstructura                # sentEstructuraLocal
    | condicional                          # sentCondicional
    | cicloPara                            # sentCicloPara
    | cicloMientras                        # sentCicloMientras
    | switchCase                           # sentSwitch
    ;
```

```
declaracionVariable:
    tipo ID dimension? (IGUAL expresion)?
    ;

dimension:
    (CORCHETE_ABRE NUMERO_ENTERO CORCHETE_CIERRA)+
    ;

tipo:
      ENTERO | FLOTANTE | CARACTER | BOOL | CADENA | ID
    ;

asignacion:
    accesoVariable IGUAL expresion
    ;

accesoVariable:
    ID (PUNTO ID | CORCHETE_ABRE expresion CORCHETE_CIERRA)*
    ;

listaValores:
    LLAVE_ABRE (expresion (COMA expresion)*)? LLAVE_CIERRA
    ;

expresion:
      PARENTESIS_ABRE expresion PARENTESIS_CIERRA            #
expParentesis
    | NEGACION expresion                                      # expNegacionLogica
    | RESTA expresion                                        # expNegativo
    | INCREMENTO ID                                           # expPreIncremento
    | DECREMENTO ID                                           # expPreDecremento
    | ID INCREMENTO                                           # expPostIncremento
    | ID DECREMENTO                                           # expPostDecremento
    | expresion op=(MULTIPLICACION|DIVISION) expresion        # expMultiplicativa
```

```
    | expresion op=(MAS|RESTA) expresion                    # expAditiva
    | expresion op=(MENOR|MAYOR|MENORIGUAL|MAYORIGUAL) expresion #
expRelacional
    | expresion op=(COMPARACION|DIFERENCIA) expresion          # expIgualdad
    | expresion op=AND expresion                               # expAnd
    | expresion op=OR expresion                                # expOr
    | llamadaFuncion                                           # expLlamada
    | leerExpr                                                 # expLeer
    | listaValores                                             # expListaValores
    | accesoVariable                                           # expAcceso
    | literal                                                  # expLiteral
    ;

llamadaFuncion:
    ID PARENTESIS_ABRE listaArgumentos? PARENTESIS_CIERRA
    ;

listaArgumentos:
    expresion (COMA expresion)*
    ;

literal:
     NUMERO_ENTERO | DECIMAL | COMILLAS | COMILLASSIMPLES |
VERDADERO | FALSO
    ;

condicional:
    SI PARENTESIS_ABRE expresion PARENTESIS_CIERRA ENTONCES bloque
    (SINO PARENTESIS_ABRE expresion PARENTESIS_CIERRA ENTONCES
bloque)*
    (CONTRARIO bloque)?
    ;

switchCase:
```

```
    ELEGIR PARENTESIS_ABRE expresion PARENTESIS_CIERRA DOS_PUNTOS
    NEWLINE+
    INDENT
        casoElegir+
        (SIEMPRE DOS_PUNTOS bloqueCaso)?
    DEDENT
    ;

casoElegir:
    CASO literal DOS_PUNTOS bloqueCaso
    ;

bloqueCaso:
    NEWLINE+
    INDENT
    sentencia+
    ROMPER?
    NEWLINE*
    DEDENT
    ;

cicloPara:
    PARA PARENTESIS_ABRE (declaracionVariable | asignacion)? PUNTO_COMA
                        expresion? PUNTO_COMA
                        expresion? PARENTESIS_CIERRA DOS_PUNTOS bloque
    ;

cicloMientras:
      MIENTRAS PARENTESIS_ABRE expresion PARENTESIS_CIERRA HACER
bloque   # cicloWhile
    | HACER DOS_PUNTOS bloque MIENTRAS PARENTESIS_ABRE expresion
PARENTESIS_CIERRA NEWLINE  # cicloDoWhile
    ;
```

```
imprimirStmt:
    IMPRIMIR PARENTESIS_ABRE expresion PARENTESIS_CIERRA
    ;

leerStmt:
    leerExpr
    ;

leerExpr:
    LEER PARENTESIS_ABRE PARENTESIS_CIERRA
    ;
```

Palabras reservadas y símbolos.

Lenguaje ZETARIANO

/*LEXER*/

```
WS:  [ \n\r\t]+ -> skip;

COMENTARIO_LINEA: '//' ~[\r\n]* -> skip;
COMENTARIO_BLOQUE: '/*' .*? '*/' -> skip;

INCREMENTO: '++';
DECREMENTO: '--';

SUMA_ASIGNACION: '+=';
RESTA_ASIGNACION: '-=';
MULTI_ASIGNACION: '*=';

MAS: '+';
RESTA: '-';
MULTIPLICACION: '*';
DIVISION: '/';
```

```
MODULO: '%';

COMILLAS: '"' (ESC | ~["\\])* '"';
fragment ESC: '\\' . ;
COMILLASSIMPLES: '\'' ~['\r\n] '\'';

MAYORIGUAL: '>=';
MENORIGUAL: '<=';
COMPARACION: '==';
DIFERENCIA: '!=';
MENOR: '<';
MAYOR: '>';

AND: '&&';
OR: '||';
NEGACION: '!';

INT: 'int';
DOUBLE: 'double';
CHAR: 'char';
BOOLEAN: 'boolean';
STRING: 'String';
NEW: 'new';
THIS: 'this';

TRUE: 'true';
FALSE: 'false';
NULL: 'null';

IF: 'if';
ELSE: 'else';

SWITCH: 'switch';
CASE: 'case';
```

BREAK: 'break';
DEFAULT: 'default';

FOR: 'for';
WHILE: 'while';
DO: 'do';
CONTINUE: 'continue';

PRINTLN: 'println';
PRINT: 'print';
READLN: 'readln';

PUBLIC: 'public';
CLASS: 'class';

VOID: 'void';
RETURN: 'return';

ID: [a-zA-Z_][a-zA-Z_0-9]*;
ENTERO: [0-9]+;
DECIMAL: [0-9]+ '.' [0-9]+;

LLAVE_ABRE: '{';
LLAVE_CIERRA: '}';
CORCHETE_ABRE: '[';
CORCHETE_CIERRA: ']';
PARENTESIS_ABRE: '(';
PARENTESIS_CIERRA: ')';
PUNTO_COMA: ';';
PUNTO: '.';
COMA: ',';
DOS_PUNTOS: ':';
IGUAL: '=';
TERNARIO: '?';

Gramática

```
/*GRAMATICA*/

program:
   creacionClase EOF
   ;

creacionClase:
   PUBLIC CLASS ID LLAVE_ABRE contenidoClase* LLAVE_CIERRA
   ;

contenidoClase:
     atributo       # contenidoAtributo
   | constructor    # contenidoConstructor
   | metodo         # contenidoMetodo
   ;

atributo:
   tipo ID (IGUAL (expresion | listaValores))? PUNTO_COMA
   ;

constructor:
   PUBLIC ID PARENTESIS_ABRE listaParametros? PARENTESIS_CIERRA
bloque
   ;

metodo:
   PUBLIC (tipo | VOID) ID PARENTESIS_ABRE listaParametros?
PARENTESIS_CIERRA bloque
   ;

listaParametros:
   parametro (COMA parametro)*
   ;
```

```
parametro:
    tipo ID
    ;

tipoBase:
      INT | DOUBLE | CHAR | STRING | BOOLEAN | ID
    ;



tipo:
    tipoBase (CORCHETE_ABRE CORCHETE_CIERRA)*
    ;

/* --------- Bloques y sentencias --------- */

cuerpo:
      bloque
    | sentencia
    ;

bloque:
    LLAVE_ABRE sentencia* LLAVE_CIERRA
    ;

sentencia:
      declaracionVariable PUNTO_COMA                    # sentDeclaracionVariable
    | asignacion PUNTO_COMA                             # sentAsignacion
    | llamadaFuncion PUNTO_COMA                         # sentLlamadaFuncion
    | llamadaMetodo PUNTO_COMA                          # sentLlamadaMetodo
    | accesoVariable (INCREMENTO | DECREMENTO) PUNTO_COMA    #
sentIncrDecrPostfijo
    | (INCREMENTO | DECREMENTO) accesoVariable PUNTO_COMA    #
sentIncrDecrPrefijo
```

```
    | imprimirStmt PUNTO_COMA                    # sentImprimir
    | readlnExpr PUNTO_COMA                      # sentReadln
    | RETURN expresion? PUNTO_COMA                    # sentReturn
    | BREAK PUNTO_COMA                           # sentBreak
    | CONTINUE PUNTO_COMA                        # sentContinue
    | condicional                          # sentCondicional
    | cicloFor                            # sentCicloFor
    | cicloWhile                           # sentCicloWhile
    | cicloDoWhile PUNTO_COMA                         # sentCicloDoWhile
    | switchCase                           # sentSwitch
    | bloque                              # sentBloqueAnidado
    ;

declaracionVariable:
    tipo ID (IGUAL (expresion | listaValores))?
    ;

listaValores:
    LLAVE_ABRE (elementoLista (COMA elementoLista)*)? LLAVE_CIERRA
    ;

elementoLista:
      expresion
    | listaValores
    ;

asignacion:
    accesoVariable op=(IGUAL | SUMA_ASIGNACION | RESTA_ASIGNACION |
MULTI_ASIGNACION) expresion
    ;

accesoVariable:
    (THIS PUNTO)? ID (PUNTO ID | CORCHETE_ABRE expresion
CORCHETE_CIERRA)*
```

```
    ;

/* --------- Condicionales --------- */

condicional:
    IF PARENTESIS_ABRE expresion PARENTESIS_CIERRA cuerpo
    (ELSE IF PARENTESIS_ABRE expresion PARENTESIS_CIERRA cuerpo)*
    (ELSE cuerpo)?
    ;

switchCase:
    SWITCH PARENTESIS_ABRE expresion PARENTESIS_CIERRA LLAVE_ABRE
        (casoSwitch | casoDefault)*
    LLAVE_CIERRA
    ;

casoSwitch:
    CASE literal DOS_PUNTOS sentencia*
    ;

casoDefault:
    DEFAULT DOS_PUNTOS sentencia*
    ;


bloqueCaso:
    sentencia*
    ;

/* --------- Ciclos --------- */

cicloFor:
    FOR PARENTESIS_ABRE (declaracionVariable | asignacion)? PUNTO_COMA
                        expresion? PUNTO_COMA
```

```
                    (asignacion | (accesoVariable (INCREMENTO | DECREMENTO)) |
((INCREMENTO | DECREMENTO) accesoVariable))?
                    PARENTESIS_CIERRA cuerpo
    ;

cicloWhile:
    WHILE PARENTESIS_ABRE expresion PARENTESIS_CIERRA cuerpo
    ;

cicloDoWhile:
    DO bloque WHILE PARENTESIS_ABRE expresion PARENTESIS_CIERRA
    ;

/* --------- Funciones especiales --------- */

imprimirStmt:
      PRINTLN PARENTESIS_ABRE expresion? PARENTESIS_CIERRA
    | PRINT PARENTESIS_ABRE expresion PARENTESIS_CIERRA
    ;

readlnExpr:
    READLN PARENTESIS_ABRE PARENTESIS_CIERRA
    ;

/* --------- Expresiones (con precedencia) --------- */

expresion:
    expresionTernaria
    ;

expresionTernaria:
      expresionOr TERNARIO expresion DOS_PUNTOS expresion     # expTernaria
    | expresionOr                                             # expSinTernaria
    ;
```

```
expresionOr:
   expresionAnd (OR expresionAnd)*
   ;

expresionAnd:
   expresionIgualdad (AND expresionIgualdad)*
   ;

expresionIgualdad:
   expresionRelacional ((COMPARACION | DIFERENCIA) expresionRelacional)*
   ;

expresionRelacional:
   expresionAditiva ((MENOR | MAYOR | MENORIGUAL | MAYORIGUAL)
expresionAditiva)*
   ;

expresionAditiva:
   expresionMultiplicativa ((MAS | RESTA) expresionMultiplicativa)*
   ;

expresionMultiplicativa:
   expresionUnaria ((MULTIPLICACION | DIVISION | MODULO) expresionUnaria)*
   ;

expresionUnaria:
     (NEGACION | RESTA | INCREMENTO | DECREMENTO) expresionUnaria   #
expUnaria
   | expresionPostfija                                  # expSinUnaria
   ;

expresionPostfija:
   expresionPrimaria ((INCREMENTO | DECREMENTO))?              # expPostfija
```

```
    ;

expresionPrimaria:
      PARENTESIS_ABRE expresion PARENTESIS_CIERRA                  #
expParentesis
    | NEW tipoBase (CORCHETE_ABRE expresion CORCHETE_CIERRA)+      #
expCrearArreglo
    | NEW ID PARENTESIS_ABRE listaArgumentos? PARENTESIS_CIERRA    #
expCrearObjeto
    | readlnExpr                                                   # expReadln
    | llamadaMetodo                                                # expLlamadaMetodo
    | llamadaFuncion                                               # expLlamadaFuncion
    | accesoVariable                                               # expAcceso
    | literal                                                      # expLiteral
    | THIS                                                         # expThis
    ;

llamadaFuncion:
    ID PARENTESIS_ABRE listaArgumentos? PARENTESIS_CIERRA
    ;

llamadaMetodo:
    (THIS | ID) (PUNTO ID)* PUNTO ID PARENTESIS_ABRE listaArgumentos?
PARENTESIS_CIERRA
    ;

listaArgumentos:
    expresion (COMA expresion)*
    ;

literal:
    ENTERO | DECIMAL | COMILLAS | COMILLASSIMPLES | TRUE | FALSE | NULL
    ;
```

Lenguaje PigLatín

```
/*LEXER*/

WS: [ \t\n\r]+ -> skip;

COMENTARIO_BLOQUE_HASH: '##' .*? '##' -> skip;
COMENTARIO_LINEA_HASH: '##' ~[\r\n]* -> skip;

COMENTARIO_LINEA:'//' ~[\r\n]* -> skip;
COMENTARIO_BLOQUE:'/*' .*? '*/' -> skip;

INCREMENTO:'++';
DECREMENTO:'--';

MAS:'+';
RESTA:'-';
MULTIPLICACION:'*';
DIVISION:'/';
MODULO:'%';

MENORLEER:'<<';
MAYORIMPRIMIR:'>>';
COMILLAS:'"' (ESC | ~["\\])* '"';
fragment ESC:'\\' .;
COMILLASSIMPLES:'\'' ~['\r\n] '\'';

MAYORIGUAL:'>=';
MENORIGUAL:'<=';
COMPARACION:'==';
DIFERENCIA:'!=';
MENOR:'<';
MAYOR:'>';

AND: '&&';
```

```
OR: '||';
NEGACION: '!';

NUMEROS: 'numerus';
TEXTUM: 'textum';
DECIMALIS: 'decimalis';
LITTERA: 'littera';
BOOL: 'bool';
NIHIL: 'nihil';

VERUM: 'verum';
FALSUS: 'falsus';
NULL: 'null';

SECCIONVARIABLE: 'VARIABILES';
SECCIONMAIN: 'MAIOR';

FINIS: 'finis' | 'FINIS';
SI: 'si';
ALITER: 'aliter';
DUM: 'dum';
FACERE: 'facere';
PER: 'per';
PERGE: 'perge';
INTERRUMPE: 'interrumpe';

IMPORT: 'import';
NOVUS: 'novus';
ESTO: 'esto';
SERIES: 'series';

ID: [a-zA-Z_] [a-zA-Z_0-9]*;

DECIMAL: [0-9]+ '.' [0-9]+;
```

```
ENTERO: [0-9]+;

IGUAL: '=';
DOSPUNTOS: ':';
LLAVE_ABRE: '{';
LLAVE_CIERRA: '}';
CORCHETE_ABRE: '[';
CORCHETE_CIERRA: ']';
PARENTESIS_ABRE: '(';
PARENTESIS_CIERRA: ')';
PUNTO_COMA: ';';
PUNTO: '.';
COMA: ',';


/*GRAMATICA*/
program:
   seccionImport+
   seccionVariables?
   seccionMain
   EOF
   ;

seccionImport:
   IMPORT ID (PUNTO ID)* PUNTO ID PUNTO_COMA?
   ;

seccionVariables:
   SECCIONVARIABLE MAYOR declaracion+
   ;

declaracion:
   declaracionVariable
   | declaracionArreglo
```

```
    ;

declaracionVariable:
    ESTO ID DOSPUNTOS tipo
    (expresion | inicializacionStruct)?
    PUNTO_COMA?
    ;

tipo:
    tipoDato
    | ID
    | NOVUS ID PARENTESIS_ABRE listaArgumentos? PARENTESIS_CIERRA
    ;

tipoDato:
    NUMEROS
    | DECIMALIS
    | TEXTUM
    | LITTERA
    | BOOL
    | NIHIL
    ;

declaracionArreglo:
    SERIES ID
    (CORCHETE_ABRE ENTERO CORCHETE_CIERRA)+
    DOSPUNTOS tipo
    (
        LLAVE_ABRE listaValoresArreglo? LLAVE_CIERRA
    )?
    PUNTO_COMA
    ;

listaValoresArreglo:
```

```
    valorArreglo (COMA valorArreglo)*
    ;

valorArreglo:
    expresion
    | inicializacionStruct
    ;

inicializacionStruct:
    LLAVE_ABRE listaValoresInicializacion? LLAVE_CIERRA
    ;

listaValoresInicializacion:
    valorInicializacion (COMA valorInicializacion)*
    ;

valorInicializacion:
    expresion
    | inicializacionStruct
    ;

seccionMain:
    SECCIONMAIN MAYOR
    sentencia*
    FINIS PUNTO_COMA
    ;

sentencia:
    declaracionVariable
    | declaracionArreglo
    | asignacion
    | condicional
    | cicloDum
    | cicloFacere
```

```
    | cicloPer
    | imprimir
    | leer
    | llamadaFuncionSentencia
    | llamadaMetodoSentencia
    | incrementoDecremento
    | PERGE PUNTO_COMA
    | INTERRUMPE PUNTO_COMA
    ;

incrementoDecremento:
    ID acceso*
    (INCREMENTO | DECREMENTO)
    PUNTO_COMA
    ;

asignacion:
    ID acceso*
    IGUAL
    (expresion | inicializacionStruct)
    PUNTO_COMA
    ;

condicional:
    SI
    PARENTESIS_ABRE expresion PARENTESIS_CIERRA
    bloquePig

    (
        ALITER
        PARENTESIS_ABRE expresion PARENTESIS_CIERRA
        bloquePig
    )*
```

```
    (
        ALITER
        bloquePig
    )?

    FINIS PUNTO_COMA
    ;

bloquePig:
    LLAVE_ABRE sentencia* LLAVE_CIERRA
    ;

cicloDum:
    DUM
    PARENTESIS_ABRE expresion PARENTESIS_CIERRA
    LLAVE_ABRE sentencia* LLAVE_CIERRA
    FINIS PUNTO_COMA
    ;

cicloFacere:
    FACERE
    LLAVE_ABRE sentencia* LLAVE_CIERRA
    DUM
    PARENTESIS_ABRE expresion PARENTESIS_CIERRA
    PUNTO_COMA
    ;

cicloPer:
    PER
    PARENTESIS_ABRE
    inicializacionPer
    PUNTO_COMA
    expresion
    PUNTO_COMA
```

```
    actualizacionPer
    PARENTESIS_CIERRA
    LLAVE_ABRE sentencia* LLAVE_CIERRA
    ;

inicializacionPer:
    ESTO ID DOSPUNTOS tipo expresion?
    | ID acceso* IGUAL expresion
    ;

actualizacionPer:
    ID acceso*
    (INCREMENTO | DECREMENTO)
    | ID acceso* IGUAL expresion
    ;

imprimir:
    MAYORIMPRIMIR
    expresion
    (MAYORIMPRIMIR expresion)*
    PUNTO_COMA
    ;

leer:
    (ID acceso*)?
    MENORLEER
    PUNTO_COMA?
    ;

llamadaFuncionSentencia:
    llamadaFuncion PUNTO_COMA
    ;

llamadaMetodoSentencia:
```

```
   ID acceso* PUNTO ID PARENTESIS_ABRE listaArgumentos?
PARENTESIS_CIERRA PUNTO_COMA
   ;

expresion:
   expresionOr
   ;

expresionOr:
   expresionAnd
   (OR expresionAnd)*
   ;

expresionAnd:
   expresionIgualdad
   (AND expresionIgualdad)*
   ;

expresionIgualdad:
   expresionRelacional
   ((COMPARACION | DIFERENCIA) expresionRelacional)*
   ;

expresionRelacional:
   expresionAditiva
   (
      (MENOR | MAYOR | MAYORIGUAL | MENORIGUAL)
      expresionAditiva
   )*
   ;

expresionAditiva:
   expresionMultiplicativa
   ((MAS | RESTA) expresionMultiplicativa)*
```

```
    ;

expresionMultiplicativa:
    expresionUnaria
    (
        (MULTIPLICACION | DIVISION | MODULO)
        expresionUnaria
    )*
    ;

expresionUnaria:
    (NEGACION | RESTA) expresionUnaria
    | expresionPrimaria
    ;

expresionPrimaria:
    literal
    | creacionObjeto
    | llamadaFuncion
    | ID acceso* (INCREMENTO | DECREMENTO)?
    | PARENTESIS_ABRE expresion PARENTESIS_CIERRA
    ;

creacionObjeto:
    NOVUS
    ID
    PARENTESIS_ABRE
    listaArgumentos?
    PARENTESIS_CIERRA
    ;

acceso:
    CORCHETE_ABRE expresion CORCHETE_CIERRA
    | PUNTO ID
```

```
    | PUNTO ID
      PARENTESIS_ABRE listaArgumentos? PARENTESIS_CIERRA
    ;

literal:
    ENTERO
    | DECIMAL
    | COMILLAS
    | COMILLASSIMPLES
    | VERUM
    | FALSUS
    | NULL
    ;

llamadaFuncion:
    ID
    PARENTESIS_ABRE
    listaArgumentos?
    PARENTESIS_CIERRA
    ;

listaArgumentos:
    expresion
    (COMA expresion)*
    ;
```

Tabla de compatibilidad

Tipo	textum	decimalis	numerus	littera	bool
textum	textum	textum	textum	textum	textum
decimalis	textum	decimalis	decimalis	decimalis	n/a
numerus	textum	decimalis	numerus	numerus	n/a
littera	textum	decimalis	numerus	littera	n/a
bool	textum	n/a	n/a	n/a	n/a

Por la inferencia de tipos en las concatenaciones siempre las variables de tipo textum tienen mayor prioridad sobre las demás, en el orden de la tabla es el orden de prioridad que tiene que cada una por ende las variables bool son las de menor prioridad.
Tomamos como ejemplo la tabla del lenguaje Pig Latín para la compatibilidad de tipos pero en los demás lenguajes funciona de la misma manera